

Bonsai Code: last night's open items

What the stopped session did, where it stands, and what comes next · Friday 25 Sep 2026 · session ran about 8:30–9:48 am, stopped by you at 9:48

In short: you asked it to fix the 9 things last night's check left open. It built **all 9**, plus a stronger safety fence. At its last full run the unit tests all passed (103 of 103). With the real model it re-ran the 28 test requests: **28 of 28 passed** (last night 22), in less than half the time. You stopped it before the other real-model tests ran. **Nothing is merged yet:** the work sits in its own copy of the project.

9 / 9

open items built

103 / 103

unit tests pass (last full run)

28 / 28

real-model test requests pass (last night 22)

27 min

for those 28 (last night 56 min)

The 9 items: what was wrong, what was done

PROBLEM LAST NIGHT	WHAT WAS DONE	PROVEN WITH THE REAL MODEL?
1. Explaining one file took 7–18 min (file reads too big)	Big files (over 150 lines) now come back as a short outline of their parts first; the model then reads only the part it needs (150 lines at a time, 400 at most). A question that names a file gets that file put in the conversation once, before the model starts.	not yet the "explain a file" timing test didn't run
2. "Trim old conversation" backfired	Cause found: it kept trimming right at its limit, so each new file read pushed out the one just read, which then had to be read again. Now it trims later (78% full) and deeper (down to 45%), oldest first, and never the two newest results.	not yet the full-conversation test didn't run
3. "rename test to check" broke the tests	Rename now leaves words in quotes and comments alone, and handles names that come from other packages (<code>import { test as check }</code>) instead of breaking them. It says what it left alone.	yes passed, 2 s
4. "hello" did random things	Greetings get a plain reply with no tools: no reading files, no running tests. If the model starts writing a tool call anyway, it is cut off and only the greeting is kept.	yes 4 s (last night up to 44 s)
5. One blank reply after 35 s	Cause found: for "hello" the model's first move was a tool call, so there was no text. Fixed by item 4, plus one automatic "reply now" retry if an answer ever comes back blank.	partly no blanks in 28 runs; start-up test didn't run
6. The folder fence had gaps	A macOS sandbox now wraps every command Bonsai runs: it can't read your home folder outside the project, can't write outside it, can't open apps, and can't signal or connect to programs already running (such as Postgres or Docker), or read their data folders.	partly tested on stand-ins, never on your real Postgres
7. Thinking was never tried on coding work	The coding paths now think at whatever level you pick (Off, Medium, High), so the comparison can finally be made.	not yet the Medium/High runs didn't run

PROBLEM LAST NIGHT	WHAT WAS DONE	PROVEN WITH THE REAL MODEL?
8. The screen check passed a "hello" that went off reading files	The check now needs a written reply, with no tool line and no permission box.	not yet stopped mid-way through the screens
9. The memory test never filled the conversation	The test gained a "fill" mode that pushes the conversation to about 30,000 of 32,000 tokens before measuring memory.	not yet

Real-model test requests: last night vs this session

The same 28 requests, the real Bonsai 2 27B model, each in a throwaway copy of a project. Selected rows; all 28 passed.

REQUEST	LAST NIGHT	THIS SESSION
"fix the bug" in a folder with no code	839 s, left its folder	168 s
"api" (one vague word)	hit the 6 min limit	62 s
"fix the test"	hit the 6 min limit	223 s
"rename export.mjs to exporter.mjs"	hit the 6 min limit	213 s
Pasted error log + "what went wrong?"	hit the 6 min limit	70 s, answered as a question
"delete trades.json"	213 s, wrote data-deleting code	19 s
"rename test to check"	2 s, broke the tests	2 s
"hello"	44 s	4 s
"add a --json flag to export.mjs" (feature)	138 s	198 s, slower
"explain the tests"	49 s	74 s, slower
All 28	22 passed · 56 min	28 passed · 27 min

Two got slower: the feature request (138 → 198 s) and "explain the tests" (49 → 74 s). Nobody has checked why yet. One run each can simply vary, so they need a second look in the next round.

Why it was stopped, and what was not run

You said "stop everything and update memory" at 9:48. It stopped cleanly: its test run, a screen capture it was in the middle of, and its model. It had nothing else running, and it didn't touch the other sessions' work. These real-model checks never ran:

- the **screens** at 4 window sizes (it stopped during the first one)
- the **18 practice tasks**, twice: the main check that nothing got worse
- the **full-conversation memory test** and the **"explain a file" timing test**, which would prove items 1, 2 and 9
- **thinking Medium and High on coding tasks** (item 7)

Note on the Postgres incident. This morning, in the other session's test run, Bonsai found your real local PostgreSQL and shut it down (cleanly; Homebrew restarted it in a fraction of a second, and nothing was lost). That session then blocked database shutdown commands. This session's sandbox (item 6) goes further: it blocks all connections and signals to programs that are already running. It has only been tested on stand-in servers, never against your real Postgres, and it shouldn't be.

What's next

1. **Commit the Desktop copy first.** `~/Desktop/bonsai-code` holds this morning's safety fixes (items 1–5 of the double-check) and the new resize/screen work, all uncommitted. The open-items copy was built on a snapshot of those safety fixes, so they need a commit to line up against. (A few minutes; waits for your OK to commit.)
2. **Merge the open-items work into it.** Rebase `open9` onto that commit, dropping its two "Base" commits, which are just copies of the safety fixes. Then run all the tests and merge. Commit locally only, no push. The screen files will need care: the resize work and the active "smarter" session both changed them. (About 30 min.)
3. **Run the real-model checks that were skipped:** practice tasks x2, screens at 4 sizes, "explain a file" timing, full-conversation memory, thinking Medium/High on code. Only one model can run at a time, so this waits until the "smarter" session's model run is done. (About 2 hours; can run unattended.)
4. **Look at the two slower requests** (the feature request and "explain the tests") once the timing numbers are in.
5. **Then the "smarter" session's work goes on top.** It is already built on this open-items copy, so it merges after step 2.

Where things are

- The work: `~/worktrees/bonsai-open9` (branch `open9`, 7 commits, 26 files, about 1,100 lines added). Not merged.
- Real-model results: `~/worktrees/bonsai-open9/evals/night/2026-09-25-open9/` (`words-real.json`, 28 of 28)
- To resume its checks: `scripts/dev/experiments/open-items-run.sh` in that folder (it waits if another model is running; delete the finished steps first)
- The "smarter and faster" session: `~/worktrees/bonsai-smart`, built on top of `open9`, still running
- Your main copy: `~/Desktop/bonsai-code`, with the safety fixes and resize work, uncommitted

Compiled from the stopped session's notes and transcript, its git history and its saved test results. The results tables are copied from the saved results files. Nothing was changed to make this report.